Sagar Fab International Company

Full Stack Developer Assignment - Product Design & Engineering Reasoning

---

Product Design & Engineering Reasoning Report

Role: Full Stack Developer (Ruby on Rails + Flutter)

Date: Updated July 6, 2026

1. The 3D Mahogany Bookshelf UI & Cover Generation

Standard ebook listing pages with flat text rows or plain cover icons look basic and fail to engage users. We implemented a mahogany bookshelf using dynamic rows and visual wood panels to trigger iBooks-style visual familiarity, turning library cataloging into an emotional, tactile experience. When the library is empty, the app renders three empty shelves overlayed with a transparent card, maintaining design continuity while prompting user uploads.

Technical Trade-off: Instead of trying to render the first page of a PDF as a thumbnail server-side (which requires native Unix dependencies like Ghostscript/Poppler/ImageMagick that often fail to compile on target evaluation machines), the backend generates a matching color hex gradient on save. The client app uses this start/end pair to draw dynamic glossy cover gradients instantly, ensuring 100% compilation portability.

2. "Continue Reading" Slider (Optimizing User Paths)

Ebook readers cycle between two or three books at a time, rarely finishing a document in one session. Navigating a library of 100+ items to locate active reads creates massive user friction. By caching recently opened book IDs in a SharedPreferences queue (limited to 5 unique entries) and rendering them as a horizontal slider at the top of the dashboard, users can resume reading in a single tap immediately upon opening the app.

3. Last Read Page Position Memory

Losing reading progress is highly frustrating. We cache current page indexes locally in SharedPreferences (keyed by book ID) on page changes. This represents a lightweight, high-performance approach that bypasses database writes. It guarantees that page progress restoration works fully offline and loads under 100ms.

4. Smart Offline Caching

Ebooks are frequently read on the go (trains, planes) where connections are spotty. Forcing a network request every time a book is opened slows down the app and consumes user data packages. The reader checks local device document storage first. If the binary exists, it loads offline instantly, saving data plans and battery while displaying an "Offline Mode" app header.

5. Search Debouncing (500ms Delay)

Querying the SQLite3 database on every keystroke triggers "HTTP request storms". If a user types "Gatsby" (6 characters) rapidly, it fires 6 parallel API requests, locking database transactions and causing screen flashing on slow connections. Debouncing waits for a 500ms typing pause, bundle queries, and fires a single HTTP request to optimize client and server performance.

6. Layout Constraints & Horizontal Containment

On small screen sizes (such as detail sheets or small list thumbnails < 100px), displaying title and author texts causes vertical RenderFlex overflows. We set height checks to automatically hide cover texts on smaller scales. Additionally, we replaced static Row tags with Wrap widgets in the list view to prevent horizontal pixel overflows on narrow devices.